

SSH Key Best Practices for 2025 - Using ed25519, key rotation, and other best practices

 brandonchecketts.com/archives/ssh-ed25519-key-best-practices-for-2025

Brandon

January 3, 2025

Apparently Google thinks I'm an expert at SSH Keys, so I'm providing an update to [my previous post](#) two years ago with some slight updates.

You can tell quite a bit about other IT professionals from their Public SSH Key! I often work with others and ask for their key when granting access to a machine I control. Its a negative sign when they ask how to create one. If they provide one in the PuttyGen format, I know they've been asked for their key exactly once. A 2048 bit or smaller RSA key means they haven't generated one in a long time. If they send me an ed25519 key with a comment other than their machine name, I feel confident that they know what they are doing.

For reference, a 4096-bit RSA key will be in this format:

And for comparison, an ed25519 key looks like this:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBLURucCueNvq4hPRkLEMHdt5tj/bSbirLC0BkXrPDI
someuser@ip-172-31-74-201
```

You'll notice in both of these, the first characters contain the key type. The middle section with all of the random characters contain the base-64 encoded public key. And at the end is a comment that is intended to identify the user to whom it belongs.

The ed25519 key is much shorter than an RSA keys, so if you've never seen one before, you might think it is less secure. But this key type is newer, and uses a totally different, more complex algorithm. Although the 256-bit ed25519 key has fewer characters, it is, for all practical purposes, as secure as the 4096-bit RSA key above. The ed25519 algorithm is more computationally complex, so it requires fewer bits for a similar level of security.

The ed25519 algorithm is based on a [specific formula for an ellipse](#) instead of prime numbers like the RSA algorithm. It has been in wide use for ~10 years, is supported by all modern software, and as such is the current standard for most professional users. Creating a key is simple with the ssh-keygen command. But before jumping to the actual command, I wanted to also explain a few other tips that I use, and think others should adopt as well.

Keys should created by individuals, not issued to groups

You should never share your private key with anybody. Ever. If a key is ever shared, you have to assume that the other party can impersonate you on any system in which it is used.

I've been a part of some teams which create a new server and create a new key to access that server, and share the new key with everybody who needs to access the machine. I think this practice stems from AWS or other providers who create an SSH key for you, along with a new machine, and the user just continues the practice. I wish they'd change that.

That's the backwards way of thinking about it. Individuals should own their own keys. They should be private. And you can add multiple public keys to resources where multiple people need access. Again, I wish AWS and others will allow this more easily instead of allowing only a single key. You then revoke access by removing the public key, instead of having to re-issue a new key whenever the group changes. (Or worse, not changing the key at all!)

Rotating your SSH keys

You should rotate your SSH keys regularly. The thought process here is that if you have used the same key for a long time, and then your laptop with your private key gets lost, or your key compromised, every machine that you've been granted access to over that time is potentially at risk, because administrators are notoriously bad about revoking access. By changing out your key regularly, you limit the potential access in the case of a compromised key. Generating a new SSH key also ensures that you are using more modern algorithms and key sizes.

I like to create a new SSH key about every two years. To remind myself to do this, I embed the year I created the key within its name. My last key was created in March 2023, which I have named . I'm creating a new key now, at the beginning of 2025, which I'll name with the current year. Each time I use it, I'm reminded when I created the key, and if it gets to be around 2 years, and I have some time free, I'll create a new key. Of course I keep all of my older keys in case I need access to something I haven't accessed for a while. My ssh-agent usually has my two most recent keys loaded. If I do need to use an older one, it is enough of a process to find and use the old one, that the first thing I'll do is update my key as soon as I get into a system where an old key was needed.

Don't use the default ssh-keygen comment

I also suggest that you make the SSH key comment something meaningful. If you don't provide a comment, most ssh-keygen implementations default to your_username@your_machine name which just might be silly or meaningless. In a professional setting, it should clearly identify you. For example as a comment is better than me00101@billys2017_macbook_air. It should be meaningful both to you, and to whomever you are sharing it.

I mentioned including the creation month above, which I like to include in the comment because when sharing the public key, it subtly demonstrates that I am security conscious,

have rotated it recently, and I know what I'm doing. The comment at the end of the key can be changed without affecting its functionality, so if I might change the comment depending on who I'm sharing it with. When I receive a public key from somebody else that contains a generic comment, I often change the comment to include their name or email address so I can later remember to whom it belongs to.

Always use a passphrase

Your SSH key is just a tiny file on disk. If your machine is ever lost, stolen, or compromised in any way by an attacker, the file is pretty easy for them to copy. Without it being encrypted with a pass phrase, it is directly usable. And if someone has access to your SSH private key, they probably have access to your bash or terminal history and would know where to use it.

As such, it is important to protect your SSH private key with a decent pass phrase. To avoid typing your pass phrase over and over, use the [SSH-Agent](#), which will remember it for your session.

Understand and use SSH-Agent Forwarding when applicable

SSH Agent Forwarding allows you to ssh into one machine, and then transparently "forward" your SSH keys to the that machine for use authenticating into a machine past it. I most often use this when authenticating to GitHub from a remote machine. Using Agent forwarding means that I don't have to copy my SSH Private key onto the remote machine in order to authenticate to GitHub from there.

You shouldn't, however, just blindly use SSH Agent Forwarding everywhere. If you access a compromised machine where an attacker may have access to your account or to the root account, you should NOT use agent forwarding since it is possible for them to intercept your private key. I've never seen this exploited, but since it is possible, you should only use SSH Agent Forwarding to systems which you trust.

The ssh-keygen Command

With all of the above context, this is the command you should use to create your ed25519 key:

```
ssh-keygen -t ed25519 -f ~/.ssh/your-key-filename -C "your-key-comment"
```

That will ask you for a pass phrase and then show you a [randomart](#) image that represents your public key when it is created. The randomart is just a visual representation of your key so that you can see it is different from others.

```
$ ssh-keygen -t ed25519 -f ~/.ssh/brandon+2025@roundsphere.com -C  
"brandon+2025@roundsphere.com"  
Generating public/private ed25519 key pair.
```

```

Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ~/.ssh/brandon+2025@roundsphere.com
Your public key has been saved in ~/.ssh/brandon+2025@roundsphere.com.pub
The key fingerprint is:
SHA256:HiCF8gbV6DpBTC2rq2IMudwBc5+QuB9NqeGtc3pmqEY brandon+2025@roundsphere
The key's randomart image is:
+--[ED25519 256]--+
| o.o.+          |
|  * +..         |
| o 0...         |
|+ A *. .       |
|.B % . S       |
|=E* = . .      |
|=+0= .         |
|+==.=         |
|B..B          |
+-----[SHA256]-----+

```

Obsessive/Compulsive Tip

This may be taking it too far, but I like to have a memorable few digits at the end of the key so that I can confirm the key got copied correctly. One of my keys ends in 7srus, so I think of it as my “7’s ‘R’ Us” key. You can do that over and over again until you find a key that you like with this one-liner:

```
rm newkey; rm newkey.pub; ssh-keygen -t ed25519 -f ./newkey -C
"brandon+2025@roundsphere.com" -N ''; cat newkey.pub;
```

That creates a key without a passphrase, so you can do it over and over quickly until you find a public key that you “like”. Then protect it with a passphrase with the command

```
ssh-keygen -p -f newkey
```

And obviously, then you rename it from newkey and newkey.pub a more meaningful name.

Replacing your public key when you use it

As you access machines, make sure to add your new key and remove old keys from your ~/.ssh/authorized_keys file. At some point, you should remove your previous key from your ssh-agent and you’ll be forced to use the old key to get in, and replace it with the new key.

Is that complete? What other tips should others know about when creating an SSH Key in 2025 and beyond?